САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа 4

Технический дизайн микросервисной архитектуры

Выполнил:
Люсин Дмитрий
Группа: К3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Необходимо выполнить технический дизайн микросервисной архитектуры для ранее реализованного монолитного REST API приложения бронирования столиков в ресторанах.

В рамках задания требуется разделить монолит на микросервисы, придерживаясь подхода database-per-service, описать взаимодействие сервисов, разделение базы данных, межсервисные endpoints, варианты запросов и ответов, а также возможные ошибки API.

Ход работы

В качестве исходной системы рассматривался монолитный backend на Node.js/Express. В монолите были реализованы пользователи, рестораны, кухни, меню, фотографии, столики, бронирования и отзывы. Для перехода к микросервисной архитектуре система была разделена по предметным областям.

1. Выделение микросервисов

Было спроектировано следующее разделение приложения:

Микросервис	Зона ответственности	База данных
API Gateway	Единая точка входа, маршрутизация запросов к сервисам	нет
Auth & User Service	Регистрация, вход, JWT, профиль пользователя	user_db
Restaurant Catalog Service	Рестораны, кухни, столики, фильтрация	catalog_db
Menu Service	Категории меню и блюда	menu_db
Media Service	Фотографии ресторанов	media_db
Booking Service	Создание, просмотр и отмена бронирований	booking_db
Review Service	Отзывы пользователей о ресторанах	review_db

На рисунке 1 представлена схема микросервисной архитектуры.

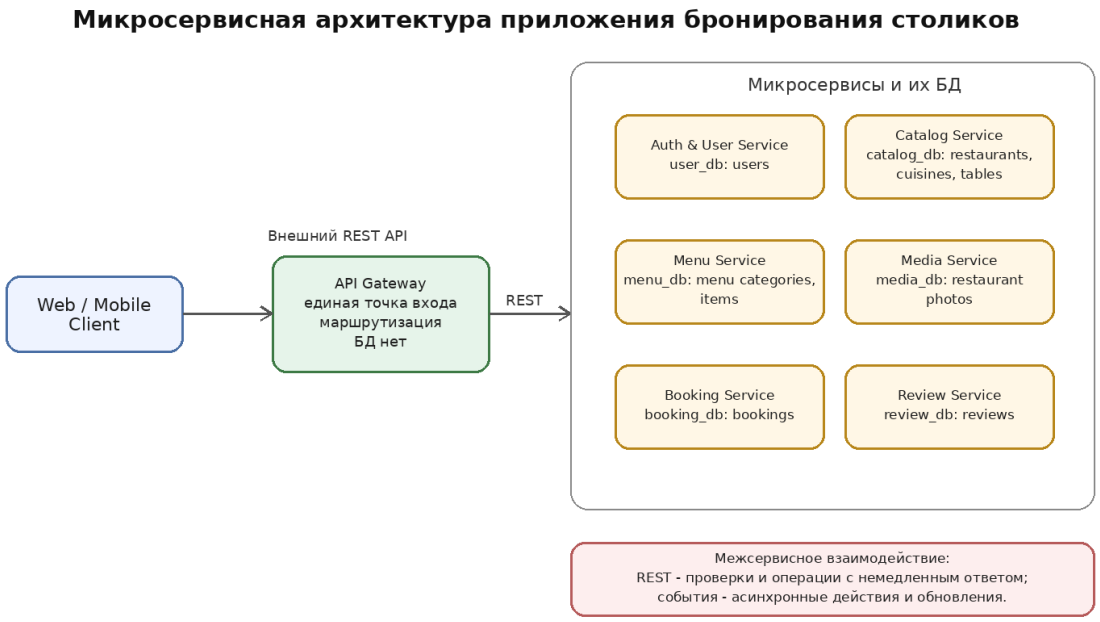


Рисунок 1 - Схема микросервисной архитектуры

2. Разделение базы данных

Для соблюдения принципа database-per-service каждая группа данных была передана во владение отдельному сервису. Сервисы не должны напрямую обращаться к таблицам друг друга.

Таблицы монолита	Новый сервис	Новая БД
users	Auth & User Service	user_db
restaurants, cuisines, restaurant_cuisines, restaurant_tables	Restaurant Catalog Service	catalog_db
menu_categories, menu_items	Menu Service	menu_db
restaurant_photos	Media Service	media_db
bookings	Booking Service	booking_db
reviews	Review Service	review_db

3. Взаимодействие микросервисов

Взаимодействие внешнего клиента с системой выполняется через API Gateway. Клиент не обращается напрямую к отдельным микросервисам.

Для межсервисного взаимодействия были предусмотрены два подхода:

- синхронные REST-запросы - используются, когда сервису нужен немедленный ответ, например при проверке пользователя, ресторана или столика;
- асинхронные события - используются для побочных действий, например обновления рейтинга ресторана или отправки уведомлений.

Пример создания бронирования: клиент отправляет запрос в API Gateway, далее запрос попадает в Booking Service. Booking Service проверяет пользователя через Auth & User Service,

проверяет ресторан и столик через Restaurant Catalog Service, после чего сохраняет бронирование в booking_db.

4. Основные межсервисные endpoints

Для внутреннего взаимодействия были спроектированы служебные endpoints. Они не предназначены для прямого вызова клиентом и должны быть защищены сервисным токеном.

Метод	Endpoint	Назначение
GET	/internal/users/{userId}	Проверка существования пользователя
GET	/internal/restaurants/{restaurantId}	Проверка существования ресторана
GET	/internal/restaurants/{restaurantId}/tables/{tableId}	Проверка столика и количества мест
GET	/internal/bookings/{bookingId}	Получение бронирования для других сервисов
GET	/internal/bookings/users/{userId}/restaurants/{restaurantId}/completed	Проверка права пользователя оставить отзыв

5. Возможные ошибки

Для межсервисных запросов были определены стандартные ошибки:

- 400 Bad Request - неверные параметры запроса;
- 401 Unauthorized - отсутствует или неверен сервисный токен;
- 403 Forbidden - сервис не имеет прав на выполнение операции;
- 404 Not Found - пользователь, ресторан, столик или бронирование не найдены;
- 409 Conflict - конфликт данных, например столик уже занят;
- 503 Service Unavailable - вызываемый сервис временно недоступен.

6. План перехода от монолита к микросервисам

Для разделения монолита был определен следующий план:

- выделить Auth & User Service и перенести туда пользователей и авторизацию;
- выделить Restaurant Catalog Service и перенести туда рестораны, кухни и столики;
- вынести меню в Menu Service, а фотографии в Media Service;
- вынести бронирования в Booking Service;
- вынести отзывы в Review Service;
- добавить API Gateway как единую точку входа;
- заменить прямые обращения к общей БД на REST-вызовы между сервисами;
- подготовить события для асинхронного взаимодействия.

Вывод

В ходе выполнения работы был подготовлен технический дизайн микросервисной архитектуры для приложения бронирования столиков в ресторанах. Монолит был разделен на отдельные сервисы по предметным областям: пользователи, каталог ресторанов, меню, медиа, бронирования и отзывы.

Для каждого сервиса была определена собственная база данных, что соответствует подходу database-per-service. Также были описаны основные способы взаимодействия сервисов: REST-запросы для операций с немедленным ответом и события для асинхронной обработки. В результате был сформирован упрощенный, но достаточный для защиты документ, показывающий требования и шаги перехода от монолита к микросервисам.